

Mod8_MA

December 19, 2025

```
[1]: import statsmodels.api as sm
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import sklearn.linear_model as skl
import sklearn.model_selection as skm
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import matplotlib.pyplot as plt
import scipy.stats as stats
```

```
[2]: from sklearn.model_selection import train_test_split
from sklearn.model_selection import RandomizedSearchCV, KFold, cross_val_score
from skorch import NeuralNetClassifier
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, \
    make_scorer
from sklearn.ensemble import \
    (RandomForestRegressor as RF,
     GradientBoostingRegressor as GBR,
     GradientBoostingClassifier as GBC,
     BaggingClassifier,
     RandomForestClassifier)
from scipy.stats import randint
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
```

1 1.) Load and Clean

```
[3]: file_path = r"C:\Users\jstol\Downloads\SDOH_2019_ZIPCODE_1_0 (2).xlsx"
data = pd.read_excel(file_path, sheet_name = 'Data')
data.shape
```

```
[3]: (41073, 355)
```

```
[4]: data_MA = pd.DataFrame(data[data['STATE'] == "Massachusetts"])
data = data_MA
```

```
[5]: columns_to_drop = [
    'STATEFIPS', 'STATE', 'REGION', 'TERRITORY', 'POINT_ZIP', 'YEAR'
]
data = data.drop(columns=columns_to_drop, errors = 'ignore')
data.head()
```

```
[5]:
```

	ZIPCODE	ZCTA	ACS_TOT_POP_WT_ZC	ACS_TOT_POP_US_ABOVE1_ZC	\
249	1001	1001.0	17312.0	17133.0	
250	1059	1002.0	30014.0	29866.0	
251	1002	1002.0	30014.0	29866.0	
252	1004	1002.0	30014.0	29866.0	
253	1003	1003.0	11357.0	11357.0	

	ACS_TOT_POP_ABOVE5_ZC	ACS_TOT_POP_ABOVE15_ZC	ACS_TOT_POP_ABOVE16_ZC	\
249	16356.0	14526.0	14386.0	
250	29142.0	26976.0	26656.0	
251	29142.0	26976.0	26656.0	
252	29142.0	26976.0	26656.0	
253	11357.0	11357.0	11357.0	

	ACS_TOT_POP_16_19_ZC	ACS_TOT_POP_ABOVE25_ZC	\
249	405.0	13291.0	
250	3825.0	14069.0	
251	3825.0	14069.0	
252	3825.0	14069.0	
253	6008.0	105.0	

	ACS_TOT_CIVIL_POP_ABOVE18_ZC	...	CDCP_NO_PHY_ACTV_ADULT_C_ZC	\
249	14117.0	...	26.6	
250	26246.0	...	25.8	
251	26246.0	...	25.8	
252	26246.0	...	25.8	
253	11218.0	...	30.0	

	CDCP_PULMONARY_ADULT_C_ZC	POS_DIST_ED_ZP	POS_DIST_MEDSURG_ICU_ZP	\
249	6.9	3.38	3.38	

250	5.0	9.24	9.24
251	5.0	7.78	7.78
252	5.0	7.05	7.05
253	3.3	7.61	7.61

	POS_DIST_TRAUMA_ZP	POS_DIST_PED_ICU_ZP	POS_DIST Obstetrics_ZP \
249	3.52	3.52	3.38
250	21.06	21.06	9.24
251	18.13	18.13	7.78
252	17.89	17.89	7.05
253	18.96	18.96	7.61

	POS_DIST_CLINIC_ZP	POS_DIST_ALC_ZP	CEN_AIAN_NH_IND
249	2.76	3.38	0.0
250	2.86	12.85	0.0
251	0.40	13.41	0.0
252	0.51	13.03	0.0
253	0.91	14.07	0.0

[5 rows x 349 columns]

2 2.) Presplit Sanitize

```
[6]: def presplit_sanitize(df, high_missing_thresh = 0.95):
    df = df.copy()
    df.replace([np.inf, -np.inf], np.nan, inplace = True)

    all_nan_cols = df.columns[df.isna().all()]
    if len(all_nan_cols):
        print("Dropping all-NaN cols (global):", list(all_nan_cols))
        df.drop(columns=all_nan_cols, inplace = True)

    miss_ratio = df.isna().mean()
    high_miss_cols = miss_ratio[miss_ratio >= high_missing_thresh].index
    if len(high_miss_cols):
        print(f"Dropping high-missing cols ( {int(high_missing_thresh*100)}%_
↳NaN):", list(high_miss_cols))
        df.drop(columns = high_miss_cols, inplace = True)

    return df

data_clean = presplit_sanitize(data, high_missing_thresh = 0.95)
```

Dropping all-NaN cols (global): ['ACS_MEDIAN_HH_INC_NHPI_ZC',
'CCBP_TOT_GAMBLING_ZP']

Dropping high-missing cols (95% NaN): ['ACS_MEDIAN_HH_INC_AIAN_ZC',
'CCBP_TOT_CHS_ZP', 'CCBP_TOT_CFS_ZP', 'CCBP_TOT_SFS_ZP', 'CCBP_TOT_EORS_ZP',

```
'CCBP_TOT_SHELTERS_ZP']
```

3 3.) Ailment List

```
[7]: column_list_ailments = [
      'ZIPCODE',
      ↪ 'ACS_PCT_CHILD_DISAB_ZC', 'ACS_PCT_DISABLE_ZC', 'ACS_PCT_NONVET_DISABLE_18_64_ZC',
      ↪ 'ACS_PCT_VET_DISABLE_18_64_ZC', 'CDCP_ARTHRITIS_ADULT_C_ZC', 'CDCP_ASTHMA_ADULT_C_ZC',
      ↪ 'CDCP_BLOOD_MED_ADULT_C_ZC', 'CDCP_CHOLES_ADULT_C_ZC', 'CDCP_CHOLES_SCR_ADULT_C_ZC',
      'CDCP_DOCTOR_VISIT_ADULT_C_ZC', 'CDCP_KIDNEY_DISEASE_ADULT_C_ZC',
      'CDCP_NO_PHY_ACTIV_ADULT_C_ZC', 'CDCP_PULMONARY_ADULT_C_ZC'
    ]

data_2 = data_clean[column_list_ailments].copy()
```

4 4.) Standardize Ailment Columns

```
[8]: id_col = ['ZIPCODE']
cols_to_scale = [col for col in data_2.columns if col not in id_col]

scaler = StandardScaler()
data_norm = data_2.copy()

scaled_values = scaler.fit_transform(data_2[cols_to_scale])

rename_map = {col: f"norm_{col}" for col in cols_to_scale}
for i, col in enumerate(cols_to_scale):
    data_norm[f"norm_{col}"] = scaled_values[:, i]

data_norm = data_norm[id_col + list(rename_map.values())]
print(data_norm.shape)
data_norm.head()
```

```
(681, 14)
```

```
[8]: ZIPCODE  norm_ACS_PCT_CHILD_DISAB_ZC  norm_ACS_PCT_DISABLE_ZC  \
249      1001                0.528513                0.161178
250      1059                0.007191               -0.514214
251      1002                0.007191               -0.514214
252      1004                0.007191               -0.514214
253      1003               -0.898479               -1.645654

      norm_ACS_PCT_NONVET_DISABLE_18_64_ZC  norm_ACS_PCT_VET_DISABLE_18_64_ZC  \
```

249	-0.057325	-0.144760
250	-0.120622	0.425938
251	-0.120622	0.425938
252	-0.120622	0.425938
253	7.236221	NaN

	norm_CDCP_ARTHRITIS_ADULT_C_ZC	norm_CDCP_ASTHMA_ADULT_C_ZC	\
249	1.130504	0.342651	
250	-1.658584	1.051864	
251	-1.658584	1.051864	
252	-1.658584	1.051864	
253	-4.335209	4.395293	

	norm_CDCP_BLOOD_MED_ADULT_C_ZC	norm_CDCP_CHOLES_ADULT_C_ZC	\
249	0.679213	0.488329	
250	-1.628709	-2.704388	
251	-1.628709	-2.704388	
252	-1.628709	-2.704388	
253	-9.462165	-5.645710	

	norm_CDCP_CHOLES_SCR_ADULT_C_ZC	norm_CDCP_DOCTOR_VISIT_ADULT_C_ZC	\
249	0.254560	0.258715	
250	-2.405870	-1.408502	
251	-2.405870	-1.408502	
252	-2.405870	-1.408502	
253	-7.879627	-4.040950	

	norm_CDCP_KIDNEY_DISEASE_ADULT_C_ZC	norm_CDCP_NO_PHY_ACTV_ADULT_C_ZC	\
249	0.841854	0.172928	
250	-0.737605	0.023094	
251	-0.737605	0.023094	
252	-0.737605	0.023094	
253	-2.909361	0.809722	

	norm_CDCP_PULMONARY_ADULT_C_ZC
249	0.756815
250	-0.586063
251	-0.586063
252	-0.586063
253	-1.787585

5 5.) Quartile Buckets

```
[9]: id_col = ['ZIPCODE']
cols_to_bucket = [c for c in data_norm.columns if c not in id_col]

for c in cols_to_bucket:
    s = pd.to_numeric(data_norm[c], errors = 'coerce')
    q1, q2, q3 = s.quantile([0.25, 0.50, 0.75])
    b = pd.cut(
        s,
        bins = [-np.inf, q1, q2, q3, np.inf],
        labels = [1, 2, 3, 4],
        include_lowest = True
    )
    data_norm[f"bucket_{c}"] = (b.cat.codes.replace(-1, pd.NA) + 1).
    ↳astype('Int64')

print(data_norm.shape)
data_norm.head()
```

(681, 27)

```
[9]:
```

	ZIPCODE	norm_ACS_PCT_CHILD_DISAB_ZC	norm_ACS_PCT_DISABLE_ZC	\
249	1001	0.528513	0.161178	
250	1059	0.007191	-0.514214	
251	1002	0.007191	-0.514214	
252	1004	0.007191	-0.514214	
253	1003	-0.898479	-1.645654	

	norm_ACS_PCT_NONVET_DISABLE_18_64_ZC	norm_ACS_PCT_VET_DISABLE_18_64_ZC	\
249	-0.057325	-0.144760	
250	-0.120622	0.425938	
251	-0.120622	0.425938	
252	-0.120622	0.425938	
253	7.236221	NaN	

	norm_CDCP_ARTHRITIS_ADULT_C_ZC	norm_CDCP_ASTHMA_ADULT_C_ZC	\
249	1.130504	0.342651	
250	-1.658584	1.051864	
251	-1.658584	1.051864	
252	-1.658584	1.051864	
253	-4.335209	4.395293	

	norm_CDCP_BLOOD_MED_ADULT_C_ZC	norm_CDCP_CHOLES_ADULT_C_ZC	\
249	0.679213	0.488329	
250	-1.628709	-2.704388	
251	-1.628709	-2.704388	

252	-1.628709	-2.704388
253	-9.462165	-5.645710

	norm_CDCP_CHOLES_SCR_ADULT_C_ZC	...	\
249		0.254560	...
250		-2.405870	...
251		-2.405870	...
252		-2.405870	...
253		-7.879627	...

	bucket_norm_ACS_PCT_VET_DISABLE_18_64_ZC		\
249		3	
250		4	
251		4	
252		4	
253		<NA>	

	bucket_norm_CDCP_ARTHRITIS_ADULT_C_ZC		\
249		4	
250		1	
251		1	
252		1	
253		1	

	bucket_norm_CDCP_ASTHMA_ADULT_C_ZC		\
249		3	
250		4	
251		4	
252		4	
253		4	

	bucket_norm_CDCP_BLOOD_MED_ADULT_C_ZC		\
249		4	
250		1	
251		1	
252		1	
253		1	

	bucket_norm_CDCP_CHOLES_ADULT_C_ZC		\
249		3	
250		1	
251		1	
252		1	
253		1	

	bucket_norm_CDCP_CHOLES_SCR_ADULT_C_ZC		\
249		3	

250	1
251	1
252	1
253	1

	bucket_norm_CDCP_DOCTOR_VISIT_ADULT_C_ZC \
249	3
250	1
251	1
252	1
253	1

	bucket_norm_CDCP_KIDNEY_DISEASE_ADULT_C_ZC \
249	4
250	1
251	1
252	1
253	1

	bucket_norm_CDCP_NO_PHY_ACTV_ADULT_C_ZC \
249	3
250	3
251	3
252	3
253	4

	bucket_norm_CDCP_PULMONARY_ADULT_C_ZC
249	4
250	2
251	2
252	2
253	1

[5 rows x 27 columns]

```
[10]: cols_to_sum = [c for c in data_norm.columns if c.startswith("bucket_")]
data_norm["zip_bucket_score"] = data_norm[cols_to_sum].sum(axis=1, min_count=1)
data_norm_bucket = data_norm[['ZIPCODE', 'zip_bucket_score']]
data_norm_bucket.head()
```

```
[10]: ZIPCODE zip_bucket_score
249 1001 44
250 1059 27
251 1002 27
252 1004 27
253 1003 21
```



```
[11]: data_norm_bucket[data_norm_bucket['zip_bucket_score'] ==  
      ↪data_norm_bucket['zip_bucket_score'].max()]
```

```
[11]:   ZIPCODE  zip_bucket_score  
      878      2664             49
```

```
[12]: data_norm_bucket[data_norm_bucket['zip_bucket_score'] ==  
      ↪data_norm_bucket['zip_bucket_score'].min()]
```

```
[12]:   ZIPCODE  zip_bucket_score  
      735      2203             4  
      736      2201             4
```

6 6.) Merge Score and Ailments

```
[13]: data_merged = data_clean.merge(  
      data_norm_bucket[['ZIPCODE', 'zip_bucket_score']],  
      on = 'ZIPCODE',  
      how = 'left'  
      )  
  
      cols_to_drop = [  
          ↪  
          ↪'ACS_PCT_CHILD_DISAB_ZC', 'ACS_PCT_DISABLE_ZC', 'ACS_PCT_NONVET_DISABLE_18_64_ZC',  
          ↪  
          ↪'ACS_PCT_VET_DISABLE_18_64_ZC', 'CDCP_ARTHRITIS_ADULT_C_ZC', 'CDCP_ASTHMA_ADULT_C_ZC',  
          ↪  
          ↪'CDCP_BLOOD_MED_ADULT_C_ZC', 'CDCP_CHOLES_ADULT_C_ZC', 'CDCP_CHOLES_SCR_ADULT_C_ZC',  
            'CDCP_DOCTOR_VISIT_ADULT_C_ZC', 'CDCP_KIDNEY_DISEASE_ADULT_C_ZC',  
            'CDCP_NO_PHY_ACTV_ADULT_C_ZC', 'CDCP_PULMONARY_ADULT_C_ZC'  
      ]  
      data_bucket_score = data_merged.drop(columns = cols_to_drop, errors = 'ignore')  
      data_bucket_score.head()
```

```
[13]:   ZIPCODE   ZCTA  ACS_TOT_POP_WT_ZC  ACS_TOT_POP_US_ABOVE1_ZC  \  
      0      1001  1001.0             17312.0             17133.0  
      1      1059  1002.0             30014.0             29866.0  
      2      1002  1002.0             30014.0             29866.0  
      3      1004  1002.0             30014.0             29866.0  
      4      1003  1003.0             11357.0             11357.0  
  
      ACS_TOT_POP_ABOVE5_ZC  ACS_TOT_POP_ABOVE15_ZC  ACS_TOT_POP_ABOVE16_ZC  \  
      0             16356.0             14526.0             14386.0  
      1             29142.0             26976.0             26656.0  
      2             29142.0             26976.0             26656.0  
      3             29142.0             26976.0             26656.0
```

4	11357.0	11357.0	11357.0
---	---------	---------	---------

	ACS_TOT_POP_16_19_ZC	ACS_TOT_POP_ABOVE25_ZC	ACS_TOT_CIVIL_POP_ABOVE18_ZC	\
0	405.0	13291.0	14117.0	
1	3825.0	14069.0	26246.0	
2	3825.0	14069.0	26246.0	
3	3825.0	14069.0	26246.0	
4	6008.0	105.0	11218.0	

	ERS_RUCA2_2010_ZP	POS_DIST_ED_ZP	POS_DIST_MEDSURG_ICU_ZP	\
0	1.0	3.38	3.38	
1	1.0	9.24	9.24	
2	1.0	7.78	7.78	
3	1.0	7.05	7.05	
4	1.0	7.61	7.61	

	POS_DIST_TRAUMA_ZP	POS_DIST_PED_ICU_ZP	POS_DIST_OBSTETRICS_ZP	\
0	3.52	3.52	3.38	
1	21.06	21.06	9.24	
2	18.13	18.13	7.78	
3	17.89	17.89	7.05	
4	18.96	18.96	7.61	

	POS_DIST_CLINIC_ZP	POS_DIST_ALC_ZP	CEN_AIAN_NH_IND	zip_bucket_score
0	2.76	3.38	0.0	44
1	2.86	12.85	0.0	27
2	0.40	13.41	0.0	27
3	0.51	13.03	0.0	27
4	0.91	14.07	0.0	21

[5 rows x 329 columns]

7 7.) PCA

```
[14]: ailment_cols = [
    ↪ 'ACS_PCT_CHILD_DISAB_ZC', 'ACS_PCT_DISABLE_ZC', 'ACS_PCT_NONVET_DISABLE_18_64_ZC',
    ↪ 'ACS_PCT_VET_DISABLE_18_64_ZC', 'CDCP_ARTHRITIS_ADULT_C_ZC', 'CDCP_ASTHMA_ADULT_C_ZC',
    ↪ 'CDCP_BLOOD_MED_ADULT_C_ZC', 'CDCP_CHOLES_ADULT_C_ZC', 'CDCP_CHOLES_SCR_ADULT_C_ZC',
    ↪ 'CDCP_DOCTOR_VISIT_ADULT_C_ZC', 'CDCP_KIDNEY_DISEASE_ADULT_C_ZC',
    ↪ 'CDCP_NO_PHY_ACTIV_ADULT_C_ZC', 'CDCP_PULMONARY_ADULT_C_ZC'
]
ailment_cols = [c for c in ailment_cols if c in data_clean.columns]
```

```

X = data_clean[ailment_cols].apply(pd.to_numeric, errors='coerce')

pipe = Pipeline(steps = [
    ('imputer', SimpleImputer(strategy = 'median')),
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components = 3, random_state = 77))
])

pcs = pipe.fit_transform(X)
pca_df = pd.DataFrame(pcs, columns = ['PC1', 'PC2', 'PC3'], index = X.index)

data_pca = data_clean.copy()
data_pca[['PC1', 'PC2', 'PC3']] = np.nan
data_pca.loc[X.index, ['PC1', 'PC2', 'PC3']] = pca_df[['PC1', 'PC2', 'PC3']]

####
expl_var = pipe.named_steps['pca'].explained_variance_ratio_
print("Explained variance by PC:", expl_var, " | Cumulative:", expl_var.
      ↪cumsum())

loadings = pd.DataFrame(
    pipe.named_steps['pca'].components_.T,
    index = X.columns,
    columns = ['PC1', 'PC2', 'PC3']
).sort_values('PC1', key = lambda s: s.abs(), ascending=False)
print(loadings.head(10))

```

Explained variance by PC: [0.38012388 0.32629698 0.07864255] | Cumulative:
[0.38012388 0.70642087 0.78506342]

	PC1	PC2	PC3
CDCP_ARTHROSITIS_ADULT_C_ZC	-0.437939	-0.022075	0.007706
CDCP_CHOLES_ADULT_C_ZC	-0.416040	-0.133082	-0.040975
CDCP_BLOOD_MED_ADULT_C_ZC	-0.406295	-0.126467	-0.099342
CDCP_KIDNEY_DISEASE_ADULT_C_ZC	-0.361484	0.231098	-0.137855
CDCP_DOCTOR_VISIT_ADULT_C_ZC	-0.328798	-0.267033	0.030628
CDCP_PULMONARY_ADULT_C_ZC	-0.328261	0.288871	-0.060932
CDCP_CHOLES_SCR_ADULT_C_ZC	-0.269302	-0.362600	0.091507
ACS_PCT_DISABLE_ZC	-0.168036	0.257006	0.191144
CDCP_NO_PHY_ACTV_ADULT_C_ZC	-0.113994	0.426303	-0.162535
ACS_PCT_NONVET_DISABLE_18_64_ZC	-0.070431	0.359671	0.229193

8 8.) Merge PC's with Bucket Score

```

[15]: data_pca_buck = data_pca.merge(
    data_bucket_score[['ZIPCODE', 'zip_bucket_score']],
    on = 'ZIPCODE',
    how = 'left'

```

```
)
data_pca_buck[['PC1','PC2','PC3','zip_bucket_score']].corr()

data_pca_buck = data_pca_buck.drop(columns = ailment_cols, errors = 'ignore')
data_pca_buck.head()
```

```
[15]:
```

	ZIPCODE	ZCTA	ACS_TOT_POP_WT_ZC	ACS_TOT_POP_US_ABOVE1_ZC	\
0	1001	1001.0	17312.0	17133.0	
1	1059	1002.0	30014.0	29866.0	
2	1002	1002.0	30014.0	29866.0	
3	1004	1002.0	30014.0	29866.0	
4	1003	1003.0	11357.0	11357.0	

	ACS_TOT_POP_ABOVE5_ZC	ACS_TOT_POP_ABOVE15_ZC	ACS_TOT_POP_ABOVE16_ZC	\
0	16356.0	14526.0	14386.0	
1	29142.0	26976.0	26656.0	
2	29142.0	26976.0	26656.0	
3	29142.0	26976.0	26656.0	
4	11357.0	11357.0	11357.0	

	ACS_TOT_POP_16_19_ZC	ACS_TOT_POP_ABOVE25_ZC	ACS_TOT_CIVIL_POP_ABOVE18_ZC	\
0	405.0	13291.0	14117.0	
1	3825.0	14069.0	26246.0	
2	3825.0	14069.0	26246.0	
3	3825.0	14069.0	26246.0	
4	6008.0	105.0	11218.0	

...	POS_DIST_TRAUMA_ZP	POS_DIST_PED_ICU_ZP	POS_DIST_OBSTETRICS_ZP	\
0	...	3.52	3.52	3.38
1	...	21.06	21.06	9.24
2	...	18.13	18.13	7.78
3	...	17.89	17.89	7.05
4	...	18.96	18.96	7.61

	POS_DIST_CLINIC_ZP	POS_DIST_ALC_ZP	CEN_AIAN_NH_IND	PC1	PC2	\
0	2.76	3.38	0.0	-1.762600	0.411885	
1	2.86	12.85	0.0	4.202610	1.860238	
2	0.40	13.41	0.0	4.202610	1.860238	
3	0.51	13.03	0.0	4.202610	1.860238	
4	0.91	14.07	0.0	12.938207	8.911970	

	PC3	zip_bucket_score
0	0.185884	44
1	-0.295629	27
2	-0.295629	27
3	-0.295629	27
4	1.708331	21

[5 rows x 332 columns]

```
[16]: data_pca_buck[['PC1', 'PC2', 'PC3', 'zip_bucket_score']].corr()
```

```
[16]:
```

	PC1	PC2	PC3	zip_bucket_score
PC1	1.000000e+00	1.219227e-16	-1.749467e-16	-0.760995
PC2	1.219227e-16	1.000000e+00	1.784922e-16	0.403727
PC3	-1.749467e-16	1.784922e-16	1.000000e+00	-0.036933
zip_bucket_score	-7.609946e-01	4.037267e-01	-3.693256e-02	1.000000

```
[17]: # 9.) Zip_Bucket_Score Outcome Model
```

```
[18]: #drop_columns = ['zip_bucket_score', 'PC1', 'PC2', 'PC3']
#X = data_pca_buck.drop(drop_columns, axis = 1, errors = 'ignore')
#Y = data_pca_buck['zip_bucket_score']
#
#X = X.drop('ZIPCODE', axis = 1, errors = 'ignore')
#
#mask_target = Y.notna()
#X = X.loc[mask_target]
#Y = Y.loc[mask_target]
#
#n = X.shape[0]
#test_df = X.sample(n = int(n * 0.2), random_state = 77)
#x_test_idx = test_df.index
#
#x_test = X.loc[x_test_idx]
#y_test = Y.loc[x_test_idx]
#
#x_train = X.drop(x_test_idx)
#y_train = Y.drop(x_test_idx)
#
#x_train = pd.get_dummies(x_train, drop_first = True).astype(float)
#x_test = pd.get_dummies(x_test, drop_first = True).astype(float)
#x_test = x_test.reindex(columns = x_train.columns, fill_value = 0)
#
#imputer = SimpleImputer(strategy = "median")
#x_train_imp_df = pd.DataFrame(imputer.fit_transform(x_train), columns = x_train.columns, index = x_train.index)
#x_test_imp_df = pd.DataFrame(imputer.transform(x_test), columns = x_test.columns, index = x_test.index)
```

9 9.) K-fold Function

```
[19]: mae_scorer = make_scorer(mean_absolute_error, greater_is_better = False)
rmse_scorer = make_scorer(mean_squared_error, greater_is_better = False)
r2_scorer = make_scorer(r2_score)

def evaluate_with_cv(model, X, y, folds = 5):
    """
    Runs K-fold CV and prints MAE, RMSE, and R2 with mean ± std.
    Works for any regression model.
    """

    kf = KFold(n_splits=folds, shuffle=True, random_state=77)

    mae_scores = -cross_val_score(model, X, y, scoring = mae_scorer, cv = kf,
    ↪n_jobs = -1)
    rmse_scores = np.sqrt(-cross_val_score(model, X, y, scoring = rmse_scorer,
    ↪cv = kf, n_jobs = -1))
    r2_scores = cross_val_score(model, X, y, scoring = r2_scorer, cv = kf,
    ↪n_jobs = -1)

    print("==== Cross-Validation Results =====")
    print(f"MAE      : {mae_scores.mean():.4f} ± {mae_scores.std():.4f}")
    print(f"RMSE      : {rmse_scores.mean():.4f} ± {rmse_scores.std():.4f}")
    print(f"R2       : {r2_scores.mean():.4f} ± {r2_scores.std():.4f}")
    print("=====\n")

    return mae_scores, rmse_scores, r2_scores
```

10 10.) Hyperparameter Grid and Function

```
[20]: param_grid_rf = {
    'n_estimators': [300, 500, 700, 900, 1200],
    'max_depth': [None, 3, 5, 8, 10, 15, 20, 30, 40],
    'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf': [1, 2, 4, 8, 12, 20],
    'max_features': ['sqrt', 'log2', 0.3, 0.5, 0.7, 1.0],
    'bootstrap': [True, False]
}
```

```
[21]: def run_random_search(model, param_grid, X, y, n_iter=40, cv=5):
    """
    Runs RandomizedSearchCV with K-fold CV and prints best parameters.
    Returns the best tuned estimator.
    """
```

```

rs = RandomizedSearchCV(
    estimator = model,
    param_distributions = param_grid,
    n_iter = n_iter,
    scoring = 'r2',
    cv = cv,
    verbose = 2,
    n_jobs = -1,
    random_state = 77
)

rs.fit(X, y)

print("\n==== Randomized Search Results =====")
print("Best Cross-Validated R2:", rs.best_score_)
print("\nBest Parameters:")
for k, v in rs.best_params_.items():
    print(f"    {k}: {v}")
print("=====\\n")

return rs.best_estimator_

```

11 11.) Random Forest Fit and Predictions

```

[22]: rf = RandomForestRegressor(n_estimators = 600, random_state = 77, n_jobs = -1)
#rf.fit(x_train_imp_df, y_train)
#
#preds = rf.predict(x_test_imp_df)
#mae = mean_absolute_error(y_test, preds)
#rmse = mean_squared_error(y_test, preds, squared = False)
#r2 = r2_score(y_test, preds)
#
#print(f"RandomForest MAE: {mae:.4f}")
#print(f"RandomForest RMSE: {rmse:.4f}")
#print(f"RandomForest R2: {r2:.4f}")

```

```

[23]: important_params = {
    "n_estimators": rf.n_estimators,
    "max_depth": rf.max_depth,
    "max_features": rf.max_features,
    "min_samples_split": rf.min_samples_split,
    "min_samples_leaf": rf.min_samples_leaf,
    "bootstrap": rf.bootstrap
}

print(important_params)

```

```
{'n_estimators': 600, 'max_depth': None, 'max_features': 1.0,
 'min_samples_split': 2, 'min_samples_leaf': 1, 'bootstrap': True}
```

```
[24]: # 11.) Top Features
```

```
[25]: #fi = pd.DataFrame(
#      {'feature': x_train_imp_df.columns, 'importance': rf.feature_importances_}
#).sort_values('importance', ascending = False)
#
#print("\nTop 10 Most Important Features:")
#print(fi.head(10).to_string(index = False))
```

The model is repeatedly pulling information from different correlated variables that describe the same underlying concept. Thankfully Random Forests don't care about multicollinearity, because we are seeing: - income twice - age three times - insurance three times - socioeconomic proxies multiple times

12 13.) Untuned PC1 Outcome Random Forest Regression Model

```
[26]: drop_columns = ['zip_bucket_score', 'PC1', 'PC2', 'PC3']
X_2 = data_pca_buck.drop(drop_columns, axis = 1, errors = 'ignore')

Y_2 = data_pca_buck['PC1']

X_2 = X_2.drop('ZIPCODE', axis = 1, errors = 'ignore')

mask_target = Y_2.notna()
X_2 = X_2.loc[mask_target]
Y_2 = Y_2.loc[mask_target]

n = X_2.shape[0]
test_df = X_2.sample(n = int(n * 0.20), random_state=77)
x_2_test_idx = test_df.index

x_2_test = X_2.loc[x_2_test_idx]
y_2_test = Y_2.loc[x_2_test_idx]

x_2_train = X_2.drop(x_2_test_idx)
y_2_train = Y_2.drop(x_2_test_idx)

x_2_train = pd.get_dummies(x_2_train, drop_first = True).astype(float)
x_2_test = pd.get_dummies(x_2_test, drop_first = True).astype(float)

x_2_test = x_2_test.reindex(columns = x_2_train.columns, fill_value=0)
```



```
imputer = SimpleImputer(strategy = "median")
```

```
x_2_train_imp = pd.DataFrame(  
    imputer.fit_transform(x_2_train),  
    columns = x_2_train.columns,  
    index = x_2_train.index  
)
```

```
x_2_test_imp = pd.DataFrame(  
    imputer.transform(x_2_test),  
    columns = x_2_test.columns,  
    index = x_2_test.index  
)
```

```
[27]: rf_2 = RandomForestRegressor(  
        n_estimators = 600,  
        random_state = 77,  
        n_jobs = -1  
    )  
  
    rf_2.fit(x_2_train_imp, y_2_train)  
    preds_2 = rf_2.predict(x_2_test_imp)  
  
    print("Cross-validation: PC1 Model")  
    evaluate_with_cv(rf_2, x_2_train_imp, y_2_train, folds=5)
```

```
Cross-validation: PC1 Model  
===== Cross-Validation Results =====  
MAE    : 0.4939 ± 0.0775  
RMSE   : 0.7835 ± 0.1856  
R2    : 0.8744 ± 0.0477  
=====
```

```
[27]: (array([0.39380504, 0.59605069, 0.49328002, 0.42418206, 0.56235039]),  
      array([0.64853692, 0.87481303, 0.72336457, 0.57397955, 1.09684659]),  
      array([0.85987333, 0.8834343 , 0.89937723, 0.93633151, 0.79295865]))
```

```
[28]: fi_2 = pd.DataFrame(  
    {  
        'feature': x_2_train_imp.columns,  
        'importance': rf_2.feature_importances_  
    }  
)  
.sort_values('importance', ascending = False)  
  
print("\nTop 10 Most Important Features (PC1 Model):")  
print(fi_2.head(10).to_string(index = False))
```

Top 10 Most Important Features (PC1 Model):

	feature	importance
	ACS_PCT_AGE_18_44_ZC	0.507096
	ACS_PCT_AGE_ABOVE65_ZC	0.146400
	ACS_PCT_HH_ABOVE65_ZC	0.025862
ACS_PCT_HH_ALONE_ABOVE65_ZC		0.022154
	ACS_PCT_NOT_LABOR_ZC	0.019018
	ACS_PCT_AGE_50_64_ZC	0.013753
	ACS_PCT_AGE_45_64_ZC	0.013489
	ACS_PCT_AGE_ABOVE80_ZC	0.013142
ACS_MEDIAN_AGE_FEMALE_ZC		0.007567
	ACS_PCT_VACANT_HU_ZC	0.007248

13 14.) Untuned PC2 Outcome Random Forest Regression Model

```
[29]: drop_columns = ['zip_bucket_score', 'PC1', 'PC2', 'PC3']
X_3 = data_pca_buck.drop(drop_columns, axis = 1, errors = 'ignore')

Y_3 = data_pca_buck['PC2']

X_3 = X_3.drop('ZIPCODE', axis = 1, errors = 'ignore')

mask_target_3 = Y_3.notna()
X_3 = X_3.loc[mask_target_3]
Y_3 = Y_3.loc[mask_target_3]

n3 = X_3.shape[0]
test_df_3 = X_3.sample(n = int(n3 * 0.20), random_state = 77)
x_3_test_idx = test_df_3.index

x_3_test = X_3.loc[x_3_test_idx]
y_3_test = Y_3.loc[x_3_test_idx]

x_3_train = X_3.drop(x_3_test_idx)
y_3_train = Y_3.drop(x_3_test_idx)

x_3_train = pd.get_dummies(x_3_train, drop_first = True).astype(float)
x_3_test = pd.get_dummies(x_3_test, drop_first = True).astype(float)

x_3_test = x_3_test.reindex(columns = x_3_train.columns, fill_value = 0)

imputer_3 = SimpleImputer(strategy = "median")
```

```

x_3_train_imp = pd.DataFrame(
    imputer_3.fit_transform(x_3_train),
    columns = x_3_train.columns,
    index = x_3_train.index
)

x_3_test_imp = pd.DataFrame(
    imputer_3.transform(x_3_test),
    columns = x_3_test.columns,
    index = x_3_test.index
)

```

```

[30]: rf_3 = RandomForestRegressor(
        n_estimators = 600,
        random_state = 77,
        n_jobs = -1
    )

rf_3.fit(x_3_train_imp, y_3_train)
preds_3 = rf_3.predict(x_3_test_imp)

print("Cross-validation: PC2 Model")
evaluate_with_cv(rf_3, x_3_train_imp, y_3_train, folds=5)

```

```

Cross-validation: PC2 Model
===== Cross-Validation Results =====
MAE      : 0.4121 ± 0.0497
RMSE     : 0.6526 ± 0.0836
R2      : 0.8963 ± 0.0153
=====

```

```

[30]: (array([0.38282891, 0.3924343 , 0.34727809, 0.45472825, 0.48327421]),
      array([0.55308371, 0.5936337 , 0.62644753, 0.70125224, 0.78872888]),
      array([0.89429104, 0.91417356, 0.90230031, 0.9024476 , 0.86840325]))

```

```

[31]: fi_3 = pd.DataFrame(
        {
            'feature': x_3_train_imp.columns,
            'importance': rf_3.feature_importances_
        }
    ).sort_values('importance', ascending = False)

print("\nTop 10 Most Important Features (PC2 Model):")
print(fi_3.head(10).to_string(index = False))

```

Top 10 Most Important Features (PC2 Model):

	feature	importance
	ACS_PER_CAPITA_INC_ZC	0.284702
	ACS_PCT_PERSON_INC_ABOVE200_ZC	0.203353
	ACS_MEDIAN_HH_INC_ZC	0.057264
	ACS_PCT_HH_PUB_ASSIST_ZC	0.056266
	ACS_PCT_HH_FOOD_STMP_BLW_POV_ZC	0.037932
	ACS_PCT_HH_FOOD_STMP_ZC	0.035271
	ACS_MEDIAN_INC_M_ZC	0.028120
	ACS_PCT_HEALTH_INC_BELOW137_ZC	0.025888
	ACS_PCT_HEALTH_INC_ABOVE400_ZC	0.025880
	ACS_MEDIAN_NONVET_INC_ZC	0.023930

```
[32]: #residuals = y_test - preds
#
#plt.figure(figsize=(6,4))
#sns.scatterplot(x=preds, y=residuals)
#plt.axhline(0, color='red')
#plt.title("Residual Plot (Bucket Score Model)")
#plt.xlabel("Predicted")
#plt.ylabel("Residuals")
#plt.show()
#
#plt.figure(figsize=(6,4))
#sns.histplot(residuals, kde=True)
#plt.title("Residual Distribution (Bucket Score Model)")
#plt.show()
#
#sm.qqplot(residuals, line='45')
#plt.title("Q-Q Plot (Bucket Score Model)")
#plt.show()
#
#train_preds = rf.predict(x_train_imp_df)
#r2_train = r2_score(y_train, train_preds)
#r2_test = r2_score(y_test, preds)
#
#print(f"Bucket Train R²: {r2_train:.4f}")
#print(f"Bucket Test R²: {r2_test:.4f}")
```

```
[33]: residuals_2 = y_2_test - preds_2

plt.figure(figsize = (7,4))
sns.scatterplot(x = preds_2, y = residuals_2)
plt.axhline(0, color = 'red', linewidth = 2)
plt.xlabel("Predicted PC1")
plt.ylabel("Residuals")
plt.title("Residual Plot (PC1 Model)")
plt.show()
```

```

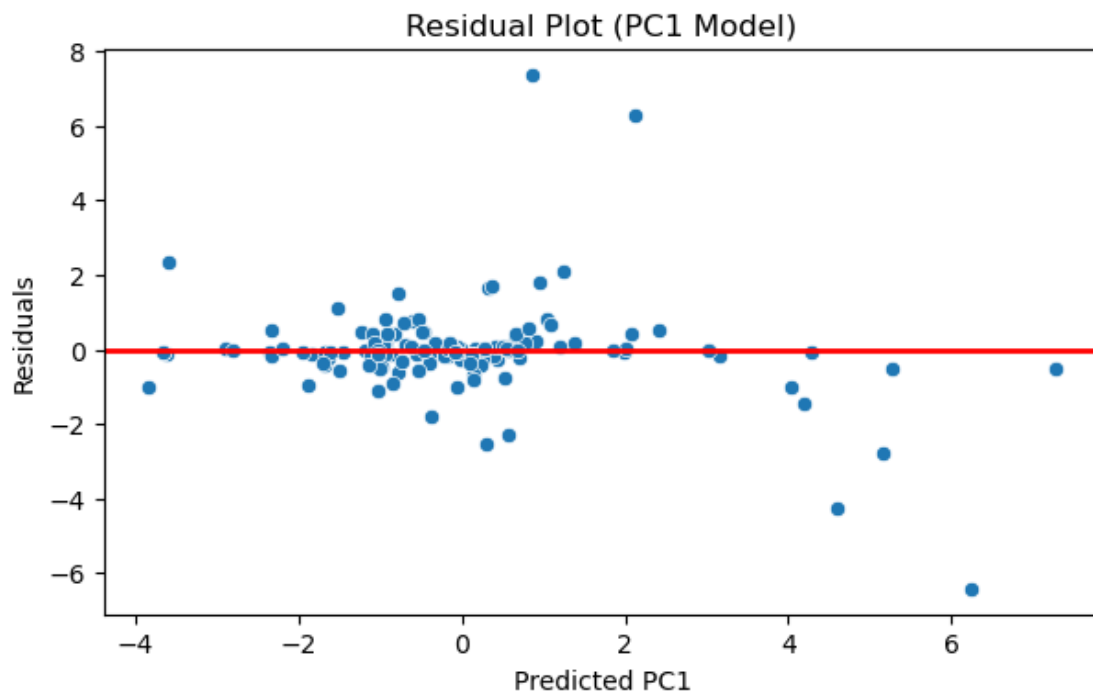
plt.figure(figsize = (7,4))
sns.histplot(residuals_2, kde = True)
plt.title("Residual Distribution (PC1 Model)")
plt.xlabel("Residual")
plt.show()

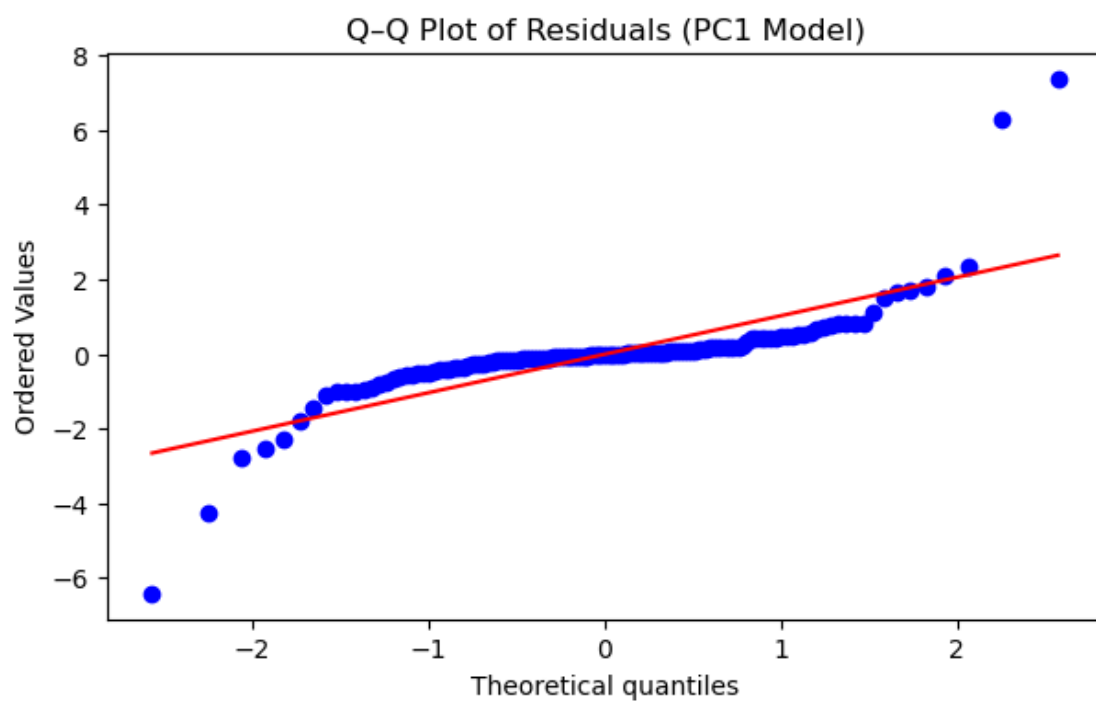
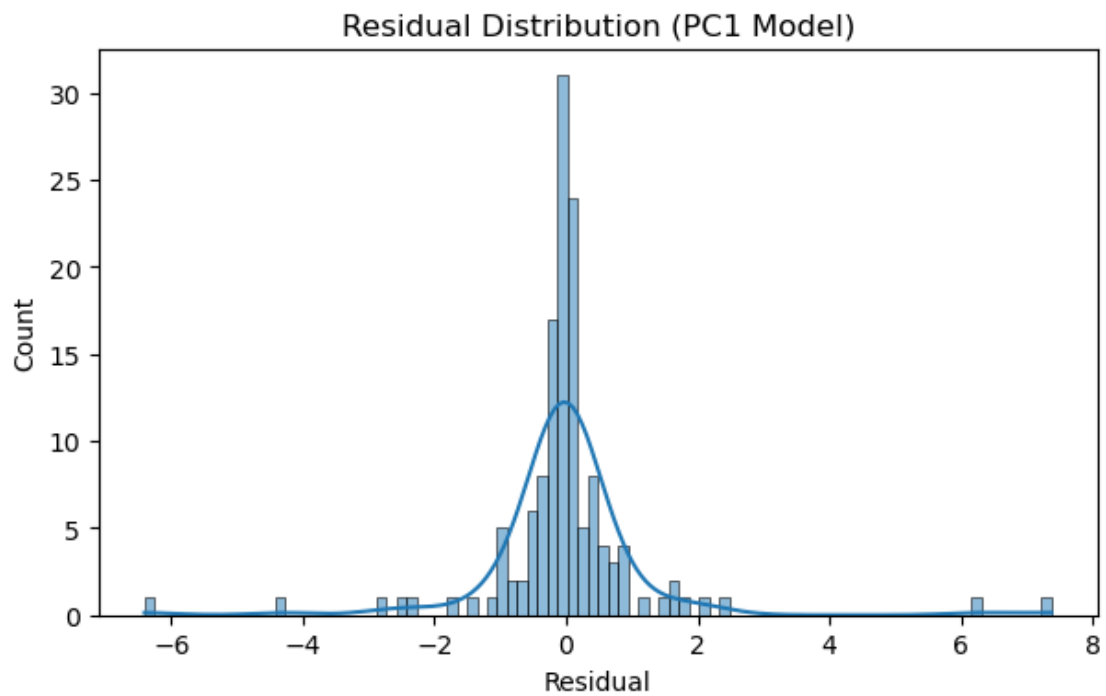
plt.figure(figsize = (7,4))
stats.probplot(residuals_2, dist = "norm", plot = plt)
plt.title("Q-Q Plot of Residuals (PC1 Model)")
plt.show()

train_preds_2 = rf_2.predict(x_2_train_imp)
r2_train_2 = r2_score(y_2_train, train_preds_2)
r2_test_2 = r2_score(y_2_test, preds_2)

print(f"PC1 Model - Train R²: {r2_train_2:.4f}")
print(f"PC1 Model - Test R²: {r2_test_2:.4f}")

```





PC1 Model - Train R^2 : 0.9845

PC1 Model - Test R^2 : 0.5992

```
[34]: residuals_3 = y_3_test - preds_3

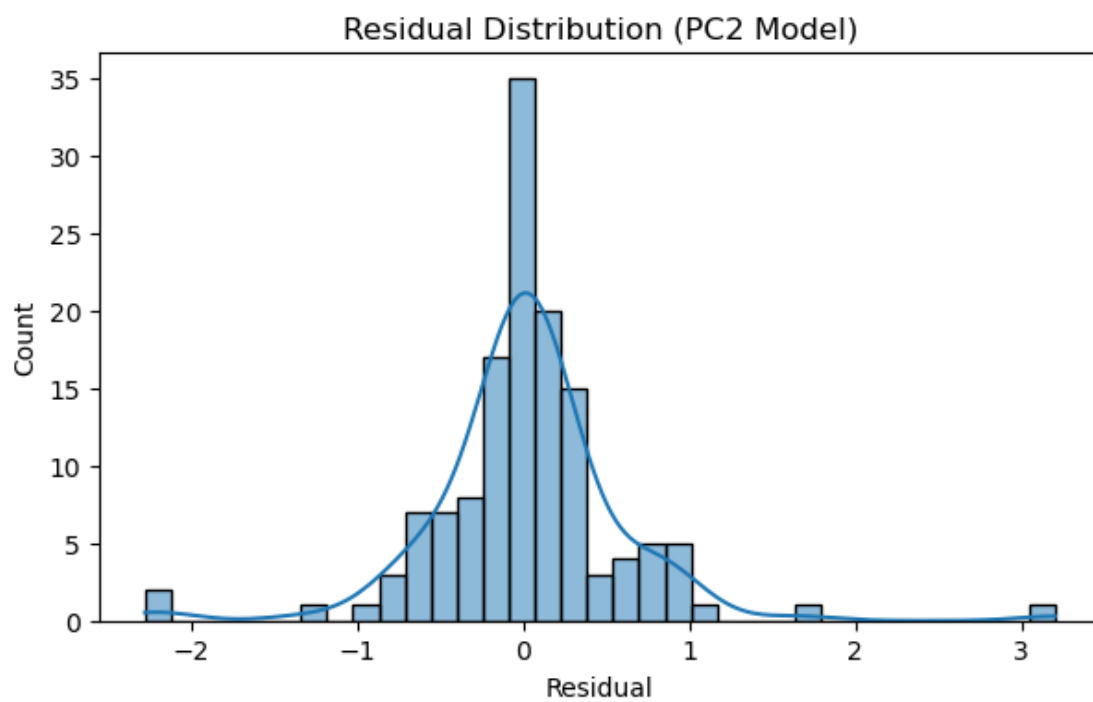
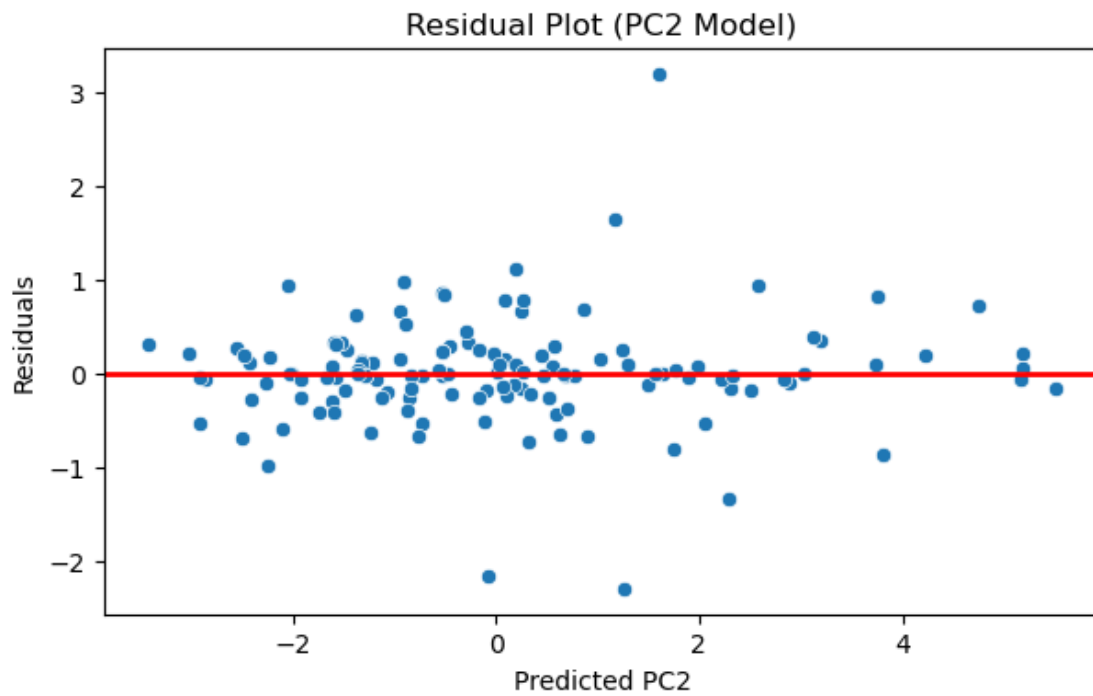
plt.figure(figsize = (7,4))
sns.scatterplot(x = preds_3, y = residuals_3)
plt.axhline(0, color = 'red', linewidth = 2)
plt.xlabel("Predicted PC2")
plt.ylabel("Residuals")
plt.title("Residual Plot (PC2 Model)")
plt.show()

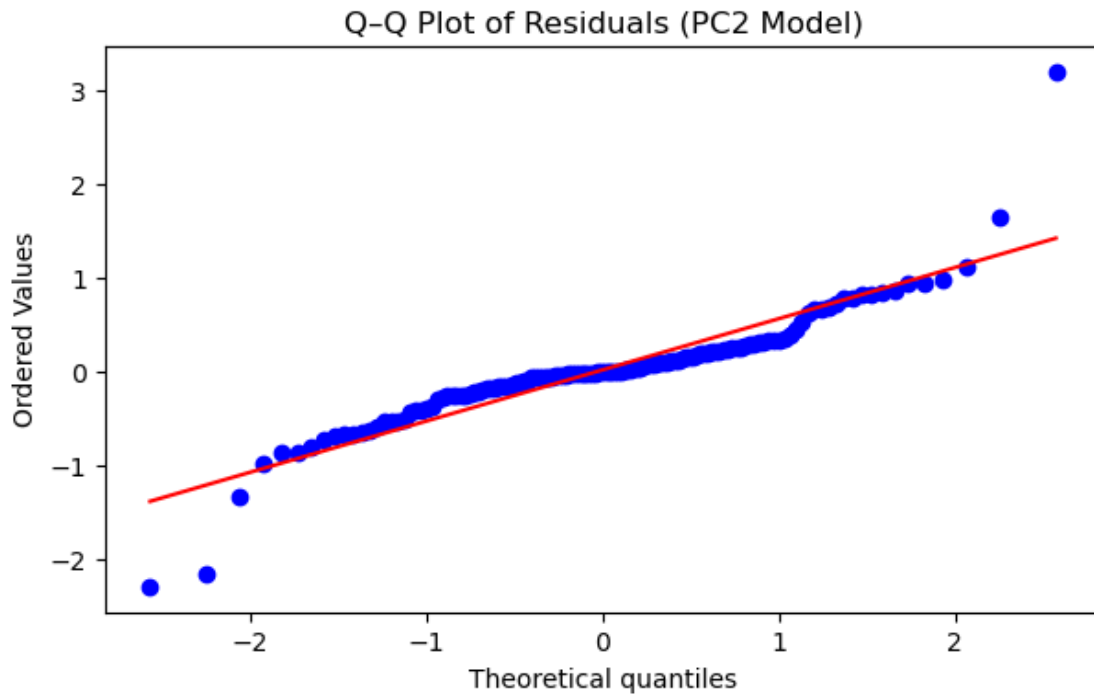
plt.figure(figsize = (7,4))
sns.histplot(residuals_3, kde = True)
plt.title("Residual Distribution (PC2 Model)")
plt.xlabel("Residual")
plt.show()

plt.figure(figsize = (7,4))
stats.probplot(residuals_3, dist = "norm", plot = plt)
plt.title("Q-Q Plot of Residuals (PC2 Model)")
plt.show()

train_preds_3 = rf_3.predict(x_3_train_imp)
r2_train_3 = r2_score(y_3_train, train_preds_3)
r2_test_3 = r2_score(y_3_test, preds_3)

print(f"PC2 Model - Train  $R^2$ : {r2_train_3:.4f}")
print(f"PC2 Model - Test  $R^2$ : {r2_test_3:.4f}")
```





PC2 Model - Train R^2 : 0.9869

PC2 Model - Test R^2 : 0.9229

```
[57]: import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats

def residual_diagnostics(model, X_train, y_train, X_test, y_test, label):
    """
    Generates:
    1) Residuals vs Predicted
    2) Residual Distribution
    3) Q-Q Plot

    Assumes model is already tuned and fitted.
    """

    model.fit(X_train, y_train)

    preds = model.predict(X_test)
    residuals = y_test - preds
```

```

plt.figure(figsize=(7,4))
sns.scatterplot(x=preds, y=residuals, alpha=0.7)
plt.axhline(0, color='red', linewidth=2)
plt.xlabel("Predicted Values")
plt.ylabel("Residuals")
plt.title(f"Residual Plot - {label}")
plt.show()

plt.figure(figsize=(7,4))
sns.histplot(residuals, kde=True, bins=30)
plt.xlabel("Residual")
plt.title(f"Residual Distribution - {label}")
plt.show()

plt.figure(figsize=(7,4))
stats.probplot(residuals, dist="norm", plot=plt)
plt.title(f"Q-Q Plot - {label}")
plt.show()

r2_train = r2_score(y_train, model.predict(X_train))
r2_test = r2_score(y_test, preds)

print(f"{label} - Train R²: {r2_train:.4f}")
print(f"{label} - Test R²: {r2_test:.4f}")
print("-" * 50)

```

14 15.) TUNED Hyperparameter Models

14.1 PC1

```

[35]: print("Running RandomizedSearchCV for PC1 Model...")

best_rf_pc1 = run_random_search(
    RandomForestRegressor(random_state = 77),
    param_grid_rf,
    x_2_train_imp,
    y_2_train
)

print("Cross-validation of Tuned PC1 Model:")
evaluate_with_cv(best_rf_pc1, x_2_train_imp, y_2_train)

```

Running RandomizedSearchCV for PC1 Model...

Fitting 5 folds for each of 40 candidates, totalling 200 fits

```
===== Randomized Search Results =====
Best Cross-Validated R2: 0.6497039068217527
```

Best Parameters:

```
n_estimators: 700
min_samples_split: 10
min_samples_leaf: 2
max_features: 0.7
max_depth: 20
bootstrap: True
```

Cross-validation of Tuned PC1 Model:

```
===== Cross-Validation Results =====
```

```
MAE      : 0.5072 ± 0.0872
```

```
RMSE     : 0.8128 ± 0.2073
```

```
R2      : 0.8649 ± 0.0539
```

```
=====
```

```
[35]: (array([0.39810153, 0.62326259, 0.50269772, 0.42634264, 0.58535177]),
       array([0.64632184, 0.91347358, 0.78116095, 0.5690012 , 1.15409913]),
       array([0.86082891, 0.87290389, 0.88265547, 0.93743116, 0.77078051]))
```

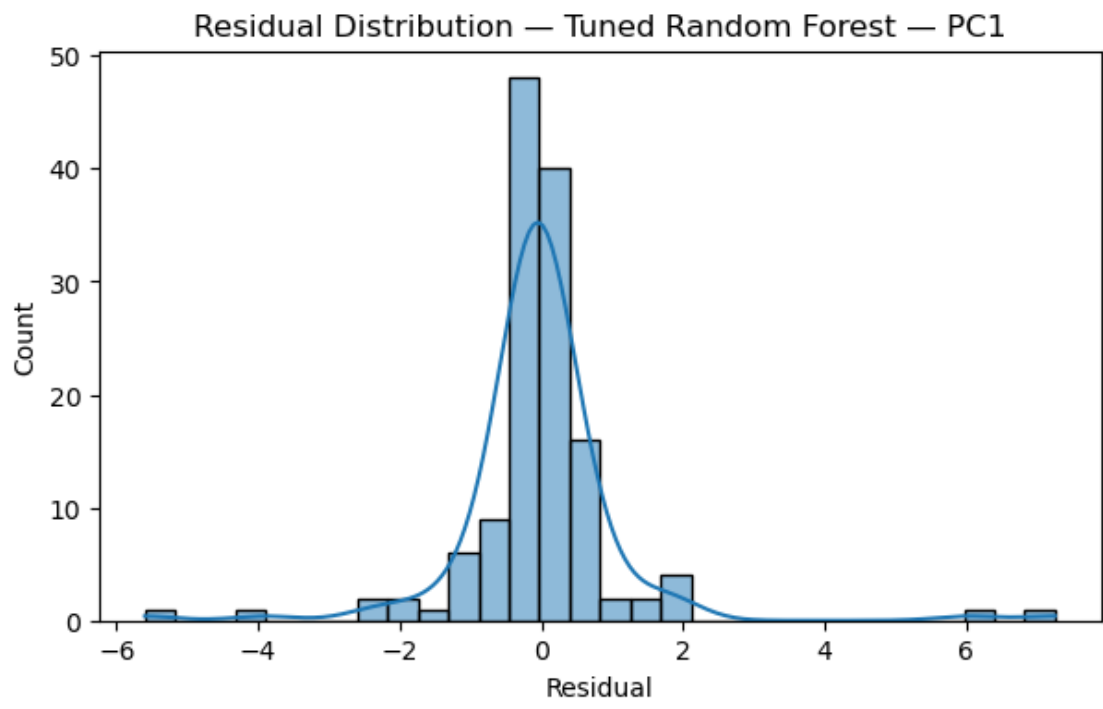
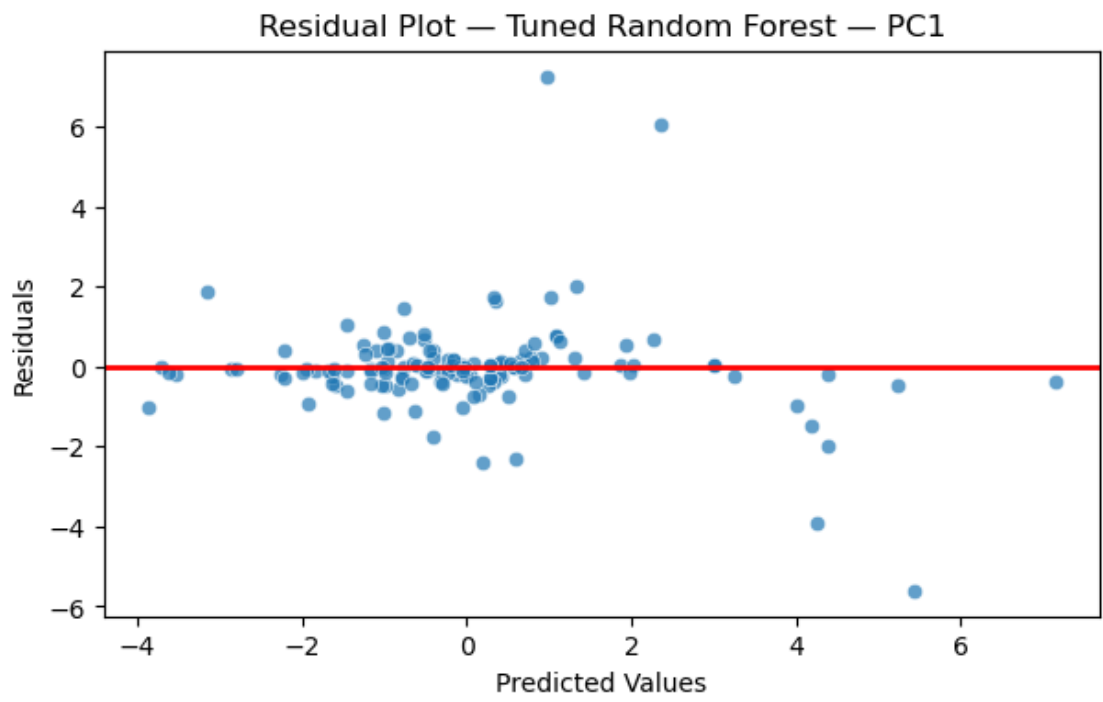
```
[36]: fi_pc1 = pd.DataFrame({
        'feature': x_2_train_imp.columns,
        'importance': best_rf_pc1.feature_importances_
    }).sort_values('importance', ascending = False)

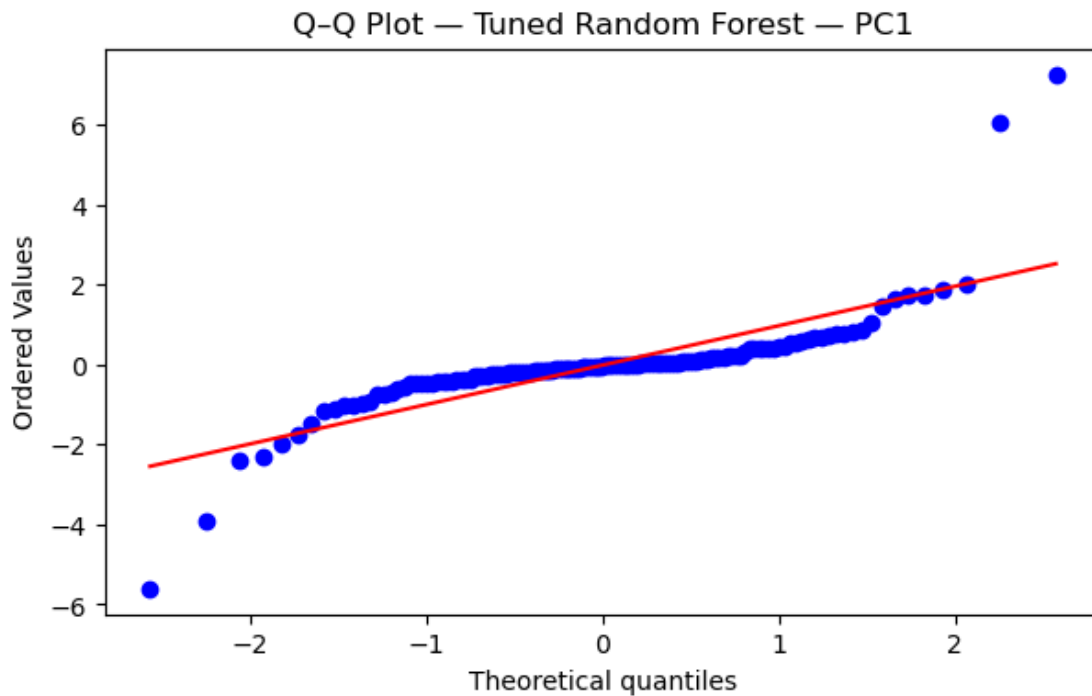
    print(fi_pc1.head(10))
```

	feature	importance
59	ACS_PCT_AGE_18_44_ZC	0.393965
63	ACS_PCT_AGE_ABOVE65_ZC	0.130197
58	ACS_PCT_AGE_18_29_ZC	0.059596
239	ACS_PCT_HH_ABOVE65_ZC	0.043081
269	ACS_PCT_DRIVE_2WORK_ZC	0.020983
240	ACS_PCT_HH_ALONE_ABOVE65_ZC	0.020208
156	ACS_PCT_NOT_LABOR_ZC	0.019196
61	ACS_PCT_AGE_45_64_ZC	0.017201
64	ACS_PCT_AGE_ABOVE80_ZC	0.015582
51	ACS_MEDIAN_AGE_FEMALE_ZC	0.015328

```
[58]: residual_diagnostics(
        model = best_rf_pc1,
        X_train = x_2_train_imp,
        y_train = y_2_train,
        X_test = x_2_test_imp,
```

```
y_test = y_2_test,  
label = "Tuned Random Forest - PC1"  
)
```





Tuned Random Forest - PC1 - Train R^2 : 0.9649

Tuned Random Forest - PC1 - Test R^2 : 0.6422

14.2 PC2

```
[37]: print("Running RandomizedSearchCV for PC2 Model...")

best_rf_pc2 = run_random_search(
    RandomForestRegressor(random_state=77),
    param_grid_rf,
    x_3_train_imp,
    y_3_train
)

print("Cross-validation of Tuned PC2 Model:")
evaluate_with_cv(best_rf_pc2, x_3_train_imp, y_3_train)
```

Running RandomizedSearchCV for PC2 Model...

Fitting 5 folds for each of 40 candidates, totalling 200 fits

===== Randomized Search Results =====

Best Cross-Validated R^2 : 0.8304668183213189

Best Parameters:

```
n_estimators: 900
min_samples_split: 20
min_samples_leaf: 2
max_features: sqrt
max_depth: 15
bootstrap: False
```

=====

Cross-validation of Tuned PC2 Model:

===== Cross-Validation Results =====

MAE : 0.4212 ± 0.0582

RMSE : 0.6619 ± 0.1150

R² : 0.8931 ± 0.0259

=====

```
[37]: (array([0.37830618, 0.42278765, 0.34133037, 0.45508331, 0.50835038]),
      array([0.53205196, 0.65089605, 0.56908248, 0.69741446, 0.8601405 ]),
      array([0.90217764, 0.8968172 , 0.91937417, 0.90351243, 0.84349491]))
```

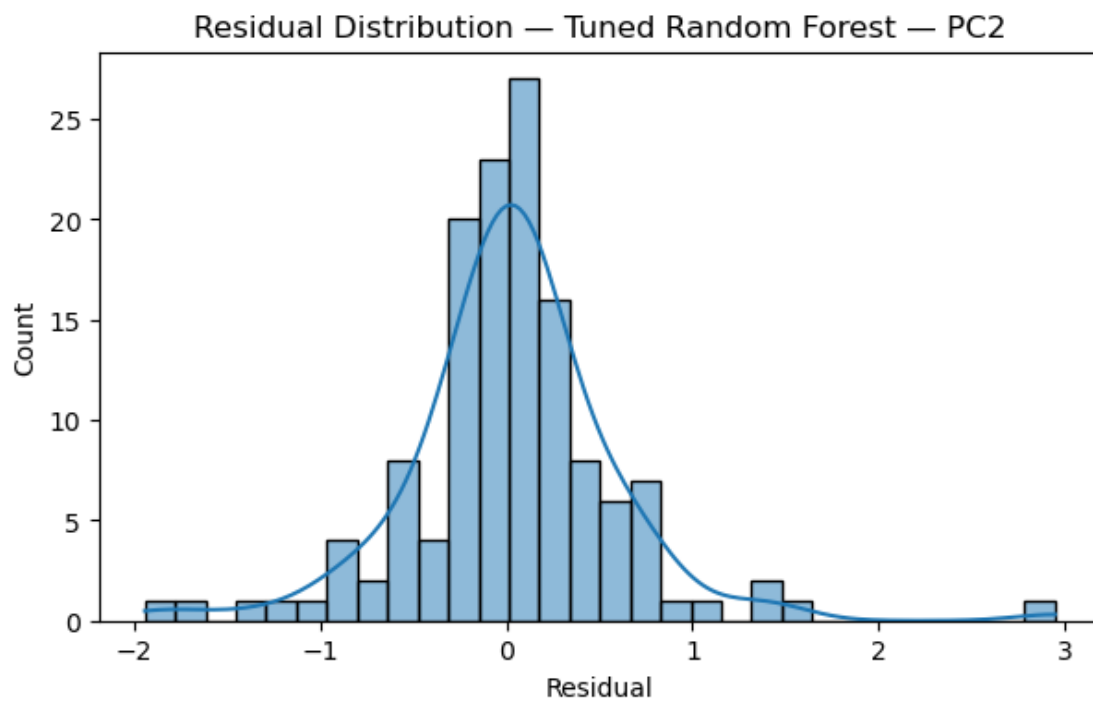
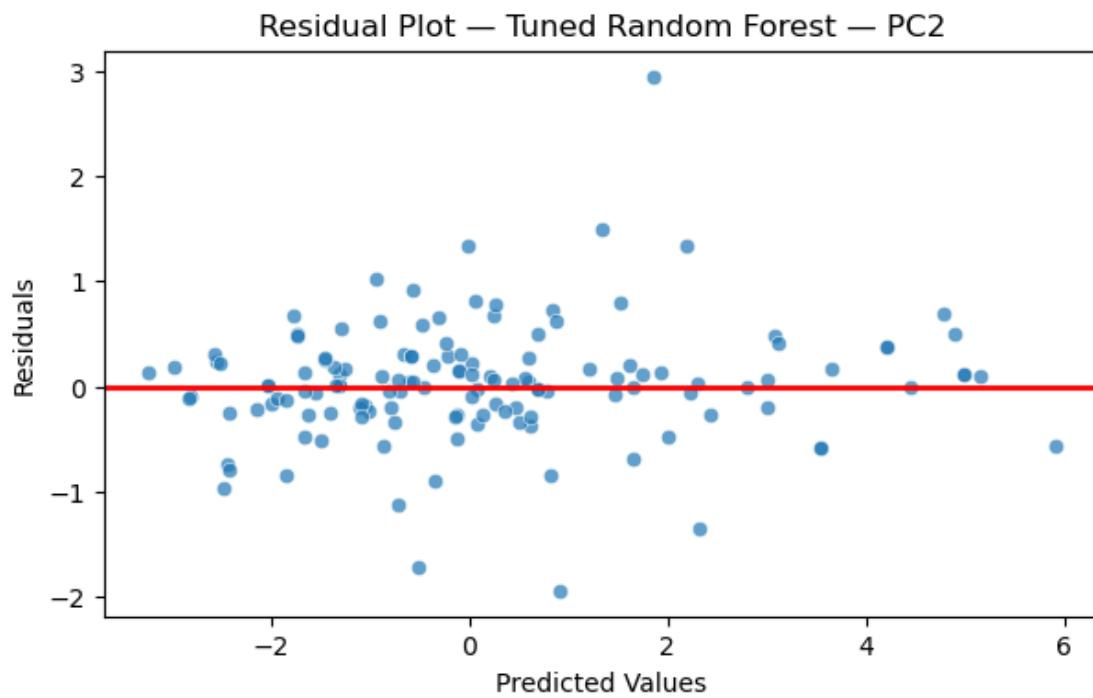
```
[38]: fi_pc2 = pd.DataFrame({
      'feature': x_3_train_imp.columns,
      'importance': best_rf_pc2.feature_importances_
    }).sort_values('importance', ascending = False)

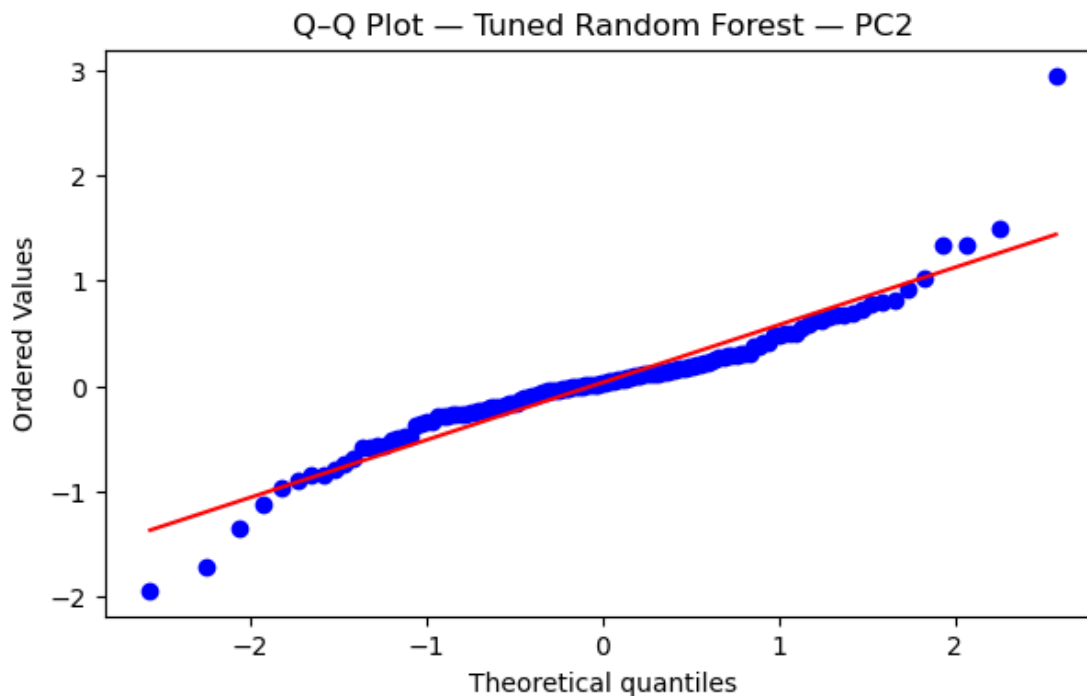
    print(fi_pc2.head(10))
```

	feature	importance
202	ACS_PER_CAPITA_INC_ZC	0.048063
200	ACS_PCT_PERSON_INC_ABOVE200_ZC	0.045982
212	ACS_PCT_HH_FOOD_STMP_ZC	0.044408
188	ACS_PCT_HEALTH_INC_ABOVE400_ZC	0.041481
185	ACS_PCT_HEALTH_INC_BELOW137_ZC	0.039358
178	ACS_MEDIAN_HH_INC_ZC	0.038707
190	ACS_PCT_HH_INC_100000_ZC	0.034460
180	ACS_MEDIAN_INC_M_ZC	0.032947
181	ACS_MEDIAN_NONVET_INC_ZC	0.031651
213	ACS_PCT_HH_FOOD_STMP_BLW_POV_ZC	0.031277

```
[59]: residual_diagnostics(
      model = best_rf_pc2,
      X_train = x_3_train_imp,
      y_train = y_3_train,
      X_test = x_3_test_imp,
      y_test = y_3_test,
      label = "Tuned Random Forest - PC2"
```

)





Tuned Random Forest - PC2 - Train R^2 : 0.9814

Tuned Random Forest - PC2 - Test R^2 : 0.9270

15 16.) PCA on Predictors (Spoiler: Much inferior)

```
[39]: from sklearn.pipeline import Pipeline
      from sklearn.compose import ColumnTransformer
      from sklearn.preprocessing import StandardScaler, OneHotEncoder
      from sklearn.impute import SimpleImputer
      from sklearn.decomposition import PCA
      from sklearn.ensemble import RandomForestRegressor
```

15.1 PC1

```
[40]: num_cols = X_2.select_dtypes(include = ['int64', 'float64']).columns
      cat_cols = X_2.select_dtypes(include = ['object', 'bool']).columns
```

```
[41]: numeric_pipeline = Pipeline([
      ('imputer', SimpleImputer(strategy = 'median')),
      ('scaler', StandardScaler()),
      ('pca', PCA(n_components = 0.95, random_state = 77))
  ])
```



```

categorical_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy = 'most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown = 'ignore'))
])

preprocess = ColumnTransformer([
    ('num', numeric_pipeline, num_cols),
    ('cat', categorical_pipeline, cat_cols)
])

```

```

[42]: rf_pca_model = Pipeline([
    ('preprocess', preprocess),
    ('rf', RandomForestRegressor(
        n_estimators = 600,
        random_state = 77,
        n_jobs = -1
    ))
])

```

```

[43]: mask = Y_2.notna()
X_2_full = X_2.loc[mask]
Y_2_full = Y_2.loc[mask]

rf_pca_model.fit(X_2_full, Y_2_full)
evaluate_with_cv(rf_pca_model, X_2_full, Y_2_full)

```

```

===== Cross-Validation Results =====
MAE    : 0.6994 ± 0.0592
RMSE   : 1.1894 ± 0.0991
R2    : 0.7004 ± 0.0841
=====

```

```

[43]: (array([0.77571363, 0.69902728, 0.75424549, 0.6428976 , 0.62509823]),
      array([1.30216527, 1.09915761, 1.31463767, 1.08711038, 1.14383276]),
      array([0.57796219, 0.78387985, 0.64165999, 0.80010549, 0.69862248]))

```

15.2 PC2

```

[44]: mask_3 = Y_3.notna()
X_3_full = X_3.loc[mask_3]
Y_3_full = Y_3.loc[mask_3]

num_cols_3 = X_3_full.select_dtypes(include = ['float64', 'int64']).columns
cat_cols_3 = X_3_full.select_dtypes(include = ['object', 'bool']).columns

```

```
[45]: numeric_pipeline_3 = Pipeline([
    ('imputer', SimpleImputer(strategy = 'median')),
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components = 0.95, random_state = 77)) # retain 95% variance
])

categorical_pipeline_3 = Pipeline([
    ('imputer', SimpleImputer(strategy = 'most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown = 'ignore'))
])

preprocess_3 = ColumnTransformer([
    ('num', numeric_pipeline_3, num_cols_3),
    ('cat', categorical_pipeline_3, cat_cols_3)
])

rf_pca_model_3 = Pipeline([
    ('preprocess', preprocess_3),
    ('rf', RandomForestRegressor(
        n_estimators = 600,
        random_state = 77,
        n_jobs = -1
    ))
])
```

```
[46]: print("PCA-on-Predictors Model for PC2:")
evaluate_with_cv(rf_pca_model_3, X_3_full, Y_3_full, folds=5)
```

```
PCA-on-Predictors Model for PC2:
===== Cross-Validation Results =====
MAE    : 0.4968 ± 0.0316
RMSE   : 0.7188 ± 0.0660
R2    : 0.8712 ± 0.0335
=====
```

```
[46]: (array([0.48115751, 0.51540501, 0.54994063, 0.46746918, 0.46978289]),
      array([0.68154896, 0.73290783, 0.83907308, 0.69271014, 0.64790825]),
      array([0.89487542, 0.82999937, 0.84336739, 0.86597083, 0.92168895]))
```

16 17.) Gradient Boosting

16.0.1 Gradient Boosting – PC1 Model (Untuned)

```
[48]: gbr_pc1 = GBR(
    random_state = 77
)
```

```
print("Cross-validation: Gradient Boosting PC1 Model")
evaluate_with_cv(gbr_pc1, x_2_train_imp, y_2_train)
```

```
Cross-validation: Gradient Boosting PC1 Model
===== Cross-Validation Results =====
MAE      : 0.4573 ± 0.0755
RMSE     : 0.7271 ± 0.1480
R2      : 0.8908 ± 0.0391
=====
```

```
[48]: (array([0.38671806, 0.54575508, 0.43476409, 0.37263722, 0.54687037]),
      array([0.65879211, 0.80251253, 0.65791407, 0.54313555, 0.97303045]),
      array([0.85540671, 0.90190566, 0.91676228, 0.94299037, 0.83706358]))
```

16.0.2 Gradient Boosting – PC2 Model (Untuned)

```
[50]: gbr_pc2 = GBR(
      random_state = 77
    )

print("Cross-validation: Gradient Boosting PC2 Model")
evaluate_with_cv(gbr_pc2, x_3_train_imp, y_3_train)
```

```
Cross-validation: Gradient Boosting PC2 Model
===== Cross-Validation Results =====
MAE      : 0.4076 ± 0.0507
RMSE     : 0.6697 ± 0.0952
R2      : 0.8903 ± 0.0216
=====
```

```
[50]: (array([0.37998998, 0.37523054, 0.34762392, 0.45682036, 0.47854803]),
      array([0.57712488, 0.60198944, 0.62543276, 0.70378835, 0.84017605]),
      array([0.88490151, 0.91174044, 0.90261658, 0.90174071, 0.85067578]))
```

16.0.3 GBR PC1 feature importance

```
[51]: fi_gbr_pc1 = pd.DataFrame({
      'feature': x_2_train_imp.columns,
      'importance': gbr_pc1.fit(x_2_train_imp, y_2_train).feature_importances_
    }).sort_values('importance', ascending = False)

print("Top 10 Gradient Boosting Features (PC1):")
print(fi_gbr_pc1.head(10).to_string(index = False))
```

```
Top 10 Gradient Boosting Features (PC1):
      feature  importance
```

ACS_PCT_AGE_18_44_ZC	0.470820
ACS_PCT_AGE_ABOVE65_ZC	0.139378
ACS_PCT_AGE_45_64_ZC	0.039638
ACS_PCT_PRIVATE_EMPL_ZC	0.036850
ACS_PCT_AGE_50_64_ZC	0.028198
ACS_PCT_API_LANG_ZC	0.020929
ACS_PCT_AGE_ABOVE80_ZC	0.020608
ACS_PCT_NOT_LABOR_ZC	0.020112
ACS_PCT_HH_ALONE_ABOVE65_ZC	0.018886
ACS_AVG_HH_SIZE_ZC	0.013609

16.0.4 GBR PC2 feature importance

```
[52]: fi_gbr_pc2 = pd.DataFrame({
        'feature': x_3_train_imp.columns,
        'importance': gbr_pc2.fit(x_3_train_imp, y_3_train).feature_importances_
    }).sort_values('importance', ascending = False)

print("Top 10 Gradient Boosting Features (PC2):")
print(fi_gbr_pc2.head(10).to_string(index = False))
```

Top 10 Gradient Boosting Features (PC2):

	feature	importance
	ACS_PER_CAPITA_INC_ZC	0.259736
	ACS_PCT_PERSON_INC_ABOVE200_ZC	0.224901
	ACS_PCT_HH_PUB_ASSIST_ZC	0.115885
	ACS_MEDIAN_HH_INC_ZC	0.066137
	ACS_PCT_HH_FOOD_STMP_BLW_POV_ZC	0.033171
	ACS_PCT_HEALTH_INC_BELOW137_ZC	0.029037
	ACS_PCT_LT_HS_ZC	0.027052
	ACS_PCT_POV_BLACK_ZC	0.026915
	ACS_PCT_POSTHS_ED_ZC	0.024556
	ACS_MEDIAN_INC_M_ZC	0.019166

16.0.5 GBR PC1 Tuning

```
[53]: param_grid_gbr = {
        'n_estimators': [100, 200, 300, 500],
        'learning_rate': [0.01, 0.05, 0.1],
        'max_depth': [2, 3, 4, 5],
        'min_samples_split': [2, 5, 10],
        'min_samples_leaf': [1, 2, 5],
        'subsample': [0.6, 0.8, 1.0]
    }
```

```
[54]: print("Running RandomizedSearchCV for Gradient Boosting PC1...")
best_gbr_pc1 = run_random_search(
    GBR(random_state = 77),
```

```

    param_grid_gbr,
    x_2_train_imp,
    y_2_train,
    n_iter = 40,
    cv = 5
)

print("Cross-validation of Tuned Gradient Boosting PC1:")
evaluate_with_cv(best_gbr_pc1, x_2_train_imp, y_2_train)

```

Running RandomizedSearchCV for Gradient Boosting PC1...
Fitting 5 folds for each of 40 candidates, totalling 200 fits

==== Randomized Search Results ====
Best Cross-Validated R^2 : 0.7135318370347858

Best Parameters:
subsample: 0.8
n_estimators: 500
min_samples_split: 10
min_samples_leaf: 1
max_depth: 3
learning_rate: 0.05
=====

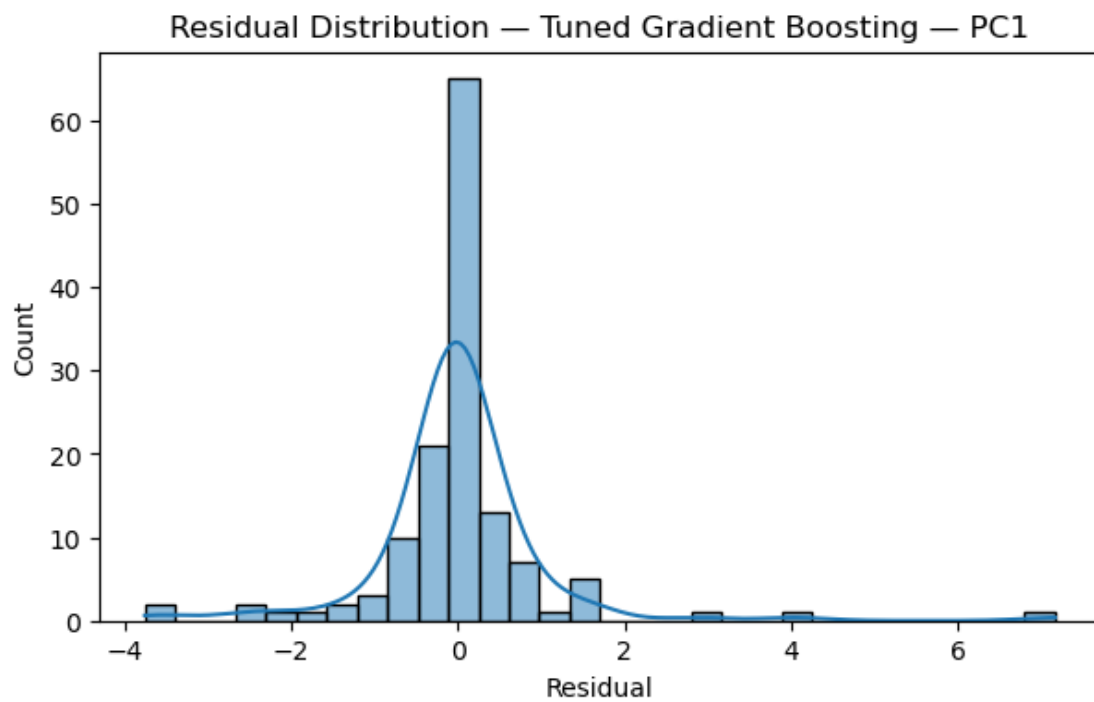
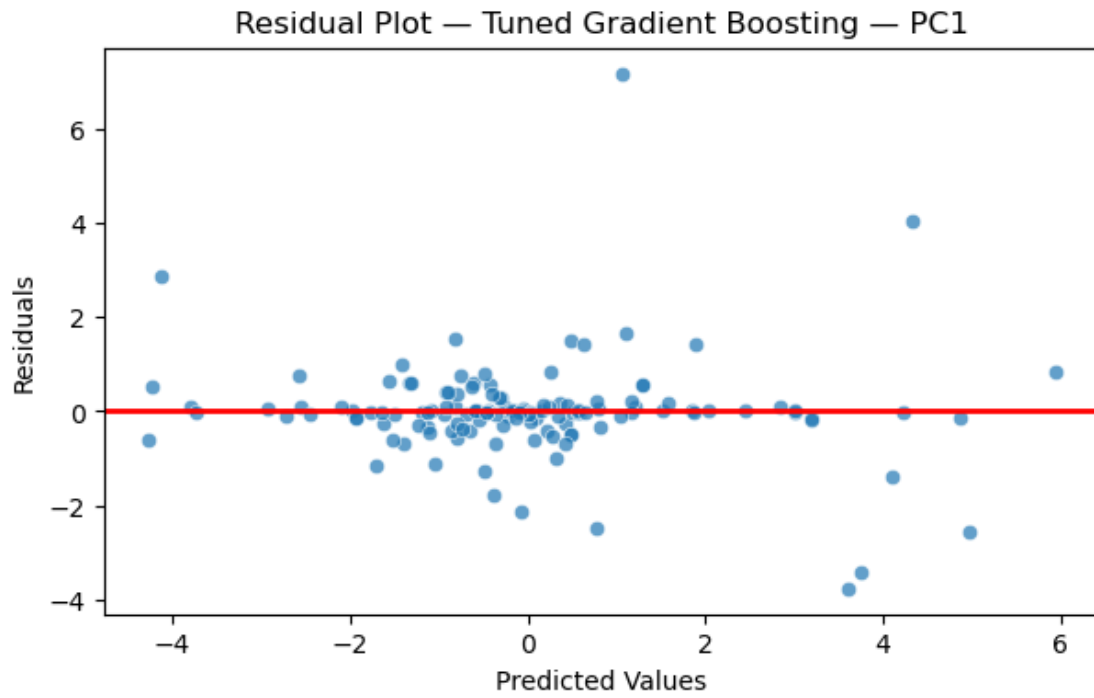
Cross-validation of Tuned Gradient Boosting PC1:
===== Cross-Validation Results =====
MAE : 0.4130 ± 0.0775
RMSE : 0.7050 ± 0.1761
 R^2 : 0.8944 ± 0.0480
=====

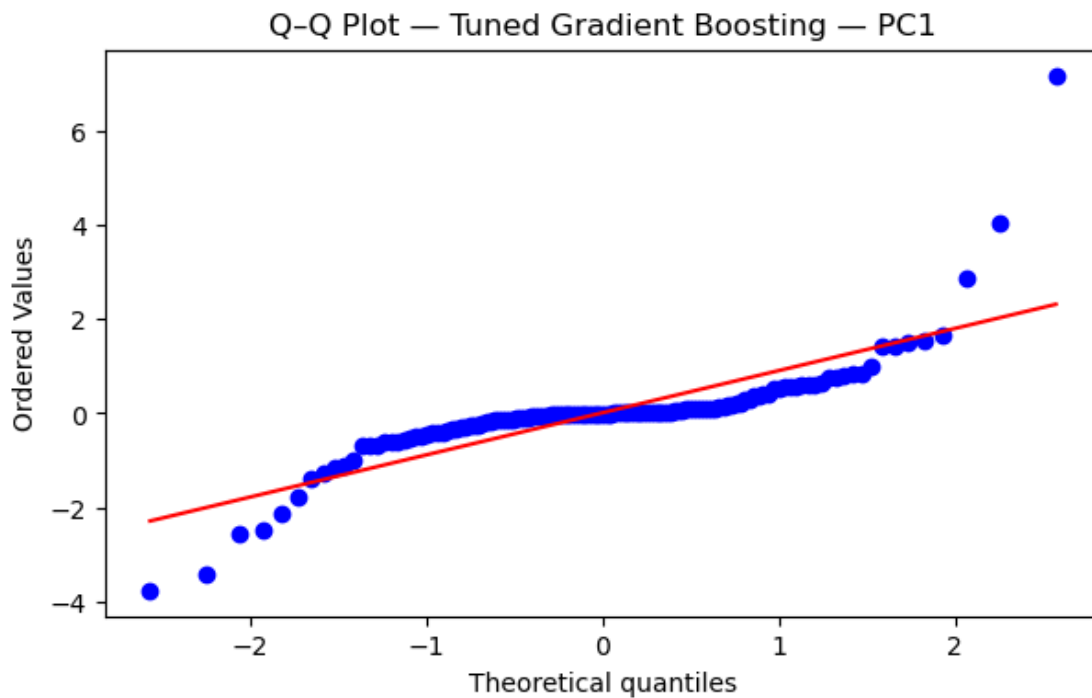
[54]: (array([0.37246172, 0.47669857, 0.37644956, 0.31256675, 0.52673022]),
array([0.67036201, 0.76004833, 0.63932723, 0.45784222, 0.99721371]),
array([0.85028333, 0.91201215, 0.92139897, 0.95948989, 0.82886384]))

```

[60]: residual_diagnostics(
    model = best_gbr_pc1,
    X_train = x_2_train_imp,
    y_train = y_2_train,
    X_test = x_2_test_imp,
    y_test = y_2_test,
    label = "Tuned Gradient Boosting - PC1"
)

```





Tuned Gradient Boosting - PC1 - Train R^2 : 0.9997

Tuned Gradient Boosting - PC1 - Test R^2 : 0.7170

16.0.6 GBR PC2 Tuning

```
[55]: print("Running RandomizedSearchCV for Gradient Boosting PC2...")
best_gbr_pc2 = run_random_search(
    GBR(random_state = 77),
    param_grid_gbr,
    x_3_train_imp,
    y_3_train,
    n_iter = 40,
    cv = 5
)

print("Cross-validation of Tuned Gradient Boosting PC2:")
evaluate_with_cv(best_gbr_pc2, x_3_train_imp, y_3_train)
```

Running RandomizedSearchCV for Gradient Boosting PC2...

Fitting 5 folds for each of 40 candidates, totalling 200 fits

===== Randomized Search Results =====

Best Cross-Validated R^2 : 0.8274198820241286

Best Parameters:

```
subsample: 0.6
n_estimators: 500
min_samples_split: 10
min_samples_leaf: 2
max_depth: 3
learning_rate: 0.1
```

=====

Cross-validation of Tuned Gradient Boosting PC2:

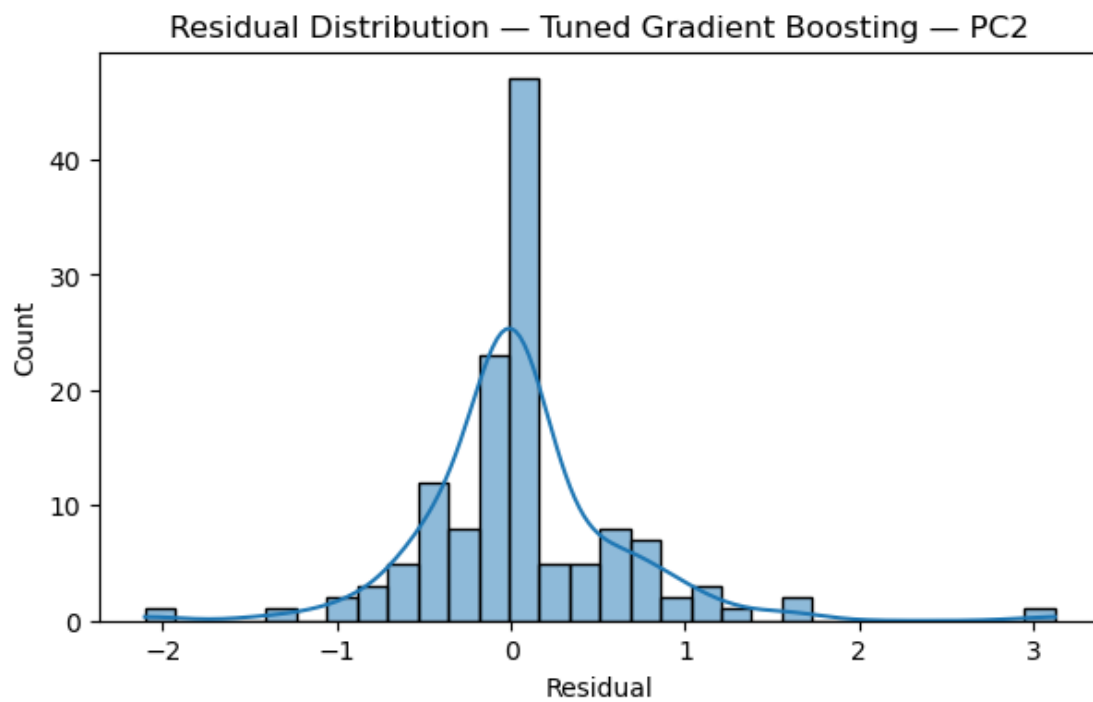
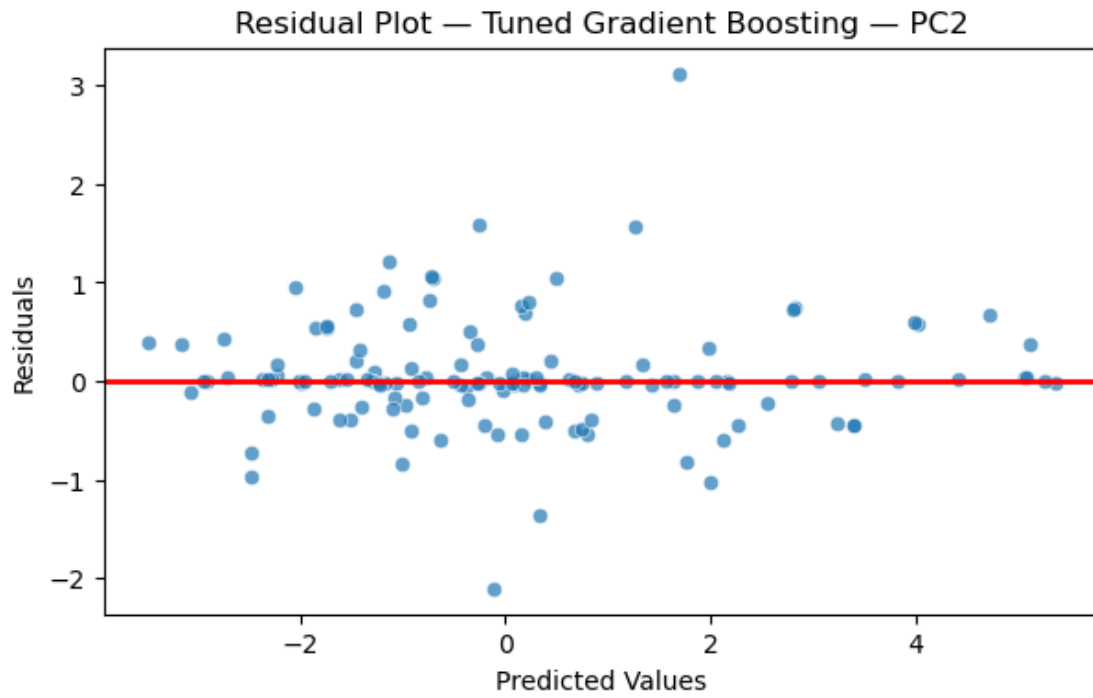
===== Cross-Validation Results =====

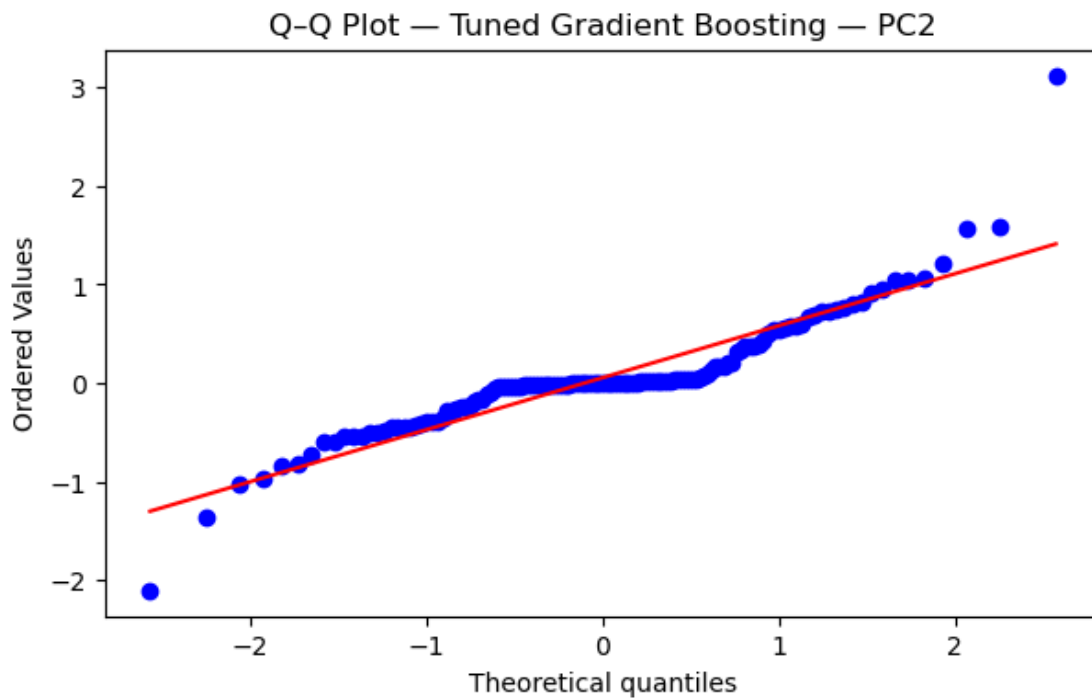
```
MAE    : 0.3687 ± 0.0548
RMSE   : 0.6334 ± 0.0909
R2    : 0.9006 ± 0.0252
```

=====

```
[55]: (array([0.37334895, 0.35354915, 0.29104036, 0.36388813, 0.46191177]),
      array([0.58967453, 0.58449852, 0.55567119, 0.62775685, 0.80939839]),
      array([0.87984142, 0.91679473, 0.92312953, 0.92182419, 0.8614156 ]))
```

```
[61]: residual_diagnostics(
      model = best_gbr_pc2,
      X_train = x_3_train_imp,
      y_train = y_3_train,
      X_test = x_3_test_imp,
      y_test = y_3_test,
      label = "Tuned Gradient Boosting - PC2"
    )
```



Tuned Gradient Boosting - PC2 - Train R^2 : 1.0000

Tuned Gradient Boosting - PC2 - Test R^2 : 0.9270

17 18.) Comparison Between Tuned RF and Tuned GBR Models

```
[56]: models = {
    "RF_PC1": best_rf_pc1,
    "GBR_PC1": best_gbr_pc1,
    "RF_PC2": best_rf_pc2,
    "GBR_PC2": best_gbr_pc2
}

for name, model in models.items():
    print(f"\n{name}")
    if "PC1" in name:
        evaluate_with_cv(model, x_2_train_imp, y_2_train)
    else:
        evaluate_with_cv(model, x_3_train_imp, y_3_train)
```

RF_PC1

===== Cross-Validation Results =====

MAE : 0.5072 ± 0.0872

RMSE : 0.8128 ± 0.2073
R² : 0.8649 ± 0.0539
=====

GBR_PC1

===== Cross-Validation Results =====
MAE : 0.4130 ± 0.0775
RMSE : 0.7050 ± 0.1761
R² : 0.8944 ± 0.0480
=====

RF_PC2

===== Cross-Validation Results =====
MAE : 0.4212 ± 0.0582
RMSE : 0.6619 ± 0.1150
R² : 0.8931 ± 0.0259
=====

GBR_PC2

===== Cross-Validation Results =====
MAE : 0.3687 ± 0.0548
RMSE : 0.6334 ± 0.0909
R² : 0.9006 ± 0.0252
=====

[]: